Give *std::optional* Range Support

Document number: P3168R2
Date: 2024-04-25
Authors: Marco Foco <marco.foco@gmail.com (mailto:marco.foco@gmail.com)>, Darius Neațu <dariusn@adobe.com (mailto:dariusn@adobe.com)>, Barry Revzin <barry.revzin@gmail.com (mailto:barry.revzin@gmail.com)>, David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>
Audience: Library Evolution

Abstract

In *P1255R12: A view of 0 or 1 elements: `views::maybe`* (https://wg21.link/P1255R12), Steve Downey explores the benefits of an optional type that models the range concept and concludes that a new optional type with range support, `std::views::maybe`, should be introduced into the standard library. This paper explores the design alternative where, instead of introducing a new type, `std::optional` is made into a range. Observing that this improves usage examples and is dramatically more straightforward, we recommend this approach over that of P1255R12.

```
// A person's attributes (e.g., eye color). All attributes are optional.
class Person {
    /* ... */
public:
    optional<string> eye_color() const;
};

vector<Person> people = ...;
```

P1255R12	This Proposal

`// Compute eye colors of 'people'.`
`vector<string> eye_colors = people`
`  | views::transform(&Person::eye_color)`
`  | views::transform(views::nullable)`
`  | views::join`
`  | ranges::to<set>()`
`  | ranges::to<vector>();`
	`// Compute eye colors of 'people'.`
	`vector<string> eye_colors = people`
	`  | views::transform(&Person::eye_color)`
	`  // no extra wrapping necessary`
	`  | views::join`
	`  | ranges::to<set>()`
	`  | ranges::to<vector>();`

Changelog

- R2:
  - Fix typo in Wording.
  - Add the LEWG Feedback#P3168R1 subsection: results of the LEWG electronic polls.
  - Add the Implementation experience#Beman.Optional26 subsection:
    - Beman.Optional26: a fully functional implementation within the Beman Project.
    - An alpha version is available on Compiler Explorer (refer to the examples).
    - Evaluated using the most recent versions of GCC and Clang compilers in accordance with the latest C++ standards.

        - Link the P3168 examples implemented in Compiler Explorer.

- R1:

    - Add the Changelog section.
    - Choice of type for iterator section discusses rationale for move from `T*` to `implementation-defined` for iterator types.
    - Additional iterator types section discusses rationale for adding only `begin()` / `end()` APIs to `std::optional`.
    - Wording updates:
        - Opt out formatter range support for optional.
        - Change the return type of iteration functions from `T*` to iterator and family.
        - Remove `{r, c}` family methods.
        - Add feature test macro.
    - Add the LEWG Feedback section: mention feedback after the Tokyo 2024 review.
    - Add the Implementation experience section.

- R0: Proposal and draft wording.

Introduction

`std::optional<T>`, a class template that "may or may not store a value of type `T` in its storage space" was introduced by Fernando Cacciola and Andrezej Krzemieński's *A proposal to add a utility class to represent optional objects (N3793)* (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html). This feature was incorporated into C++17 and has since seen frequent use in data members, function parameters, and function return types.

Ranges were introduced into C++20 with Eric Niebler et al.'s *The One Ranges Proposal (P0896R4)* (https://wg21.link/p0896r4) as a set of standard library concepts and algorithms that simplify algorithm composition and usage.

As usage experience of both `std::optional` and ranges has grown, it became apparent that treating `std::optional` objects as ranges is quite useful, especially in algorithmic contexts. Steve Downey in *A view of 0 or 1 elements: `views::maybe` (P1255R12)* (https://wg21.link/p1255r12) showcases many examples where algorithm readability improves when `std::optional` objects are treated as ranges.

Downey proposed two facilities that integrate `std::optional` and ranges:

1. `views::nullable`. A range adapter producing a view over `std::optional` objects and, more generally, any dereferencable object with a bool conversion.
2. `views::maybe`. A data type with the same semantics as `std::optional` with the some key interface differences. We will discuss these differences a little later.

Downey suggests `std::maybe_view` be used instead of `std::optional` for return types and other situations where "the value will have operations applied if present, and ignored otherwise." Furthermore, Downey claims "`std::optional` is filling too many roles" and is not "directly safe", unlike `std::maybe_view`.

While we support `views::nullable`, we find the addition of the `std::maybe_view` contrary to the principles of simplicity and genericity. Instead, we propose making `std::optional` a range.

This paper highlights the principles behind our design alternative, outlines the drawbacks of adding a new `std::optional`-like type, surveys existing practice, and demonstrates how our proposal simplifies all of P1255R12's examples.

Principles

Two design principles directly apply to the rangification of *std::optional*: genericity and simplicity.

Alex Stepanov, the progenitor of much of the C++ standard library, repeatedly emphasized the importance of genericity for software reuse. In *Fundamentals of Generic Programming* (http://stepanovpapers.com/DeSt98.pdf) he and James C. Dehnert wrote:

> Generic programming recognizes that dramatic productivity improvements must come from reuse without modification, as with the successful libraries. Breadth of use, however, must come from the separation of underlying data types, data structures, and algorithms, allowing users to combine components of each sort from either the library or their own code. Accomplishing this requires more than just simple, abstract interfaces – it requires that a wide variety of components share the same interface so that they can be substituted for one another. It is vital that we go beyond the old library model of reusing identical interfaces with pre-determined types, to one which identifies the minimal requirements on interfaces and allows reuse by similar interfaces which meet those requirements but may differ quite widely otherwise. Sharing similar interfaces across a wide variety of components requires careful identification and abstraction of the patterns of use in many programs, as well as development of techniques for effectively mapping one interface to another.

Simplicity, the second applicable principle, makes C++ easier to learn and use, increasing its applicability to different problem domains. The direction group in *Direction for ISO C++ (P2000R4)* (https://wg21.link/P2000R4) sees "C++ in danger of losing coherency due to proposals based on differing and sometimes mutually contradictory design philosophies and differing stylistic tastes." and David Sankel highlights the impact on training costs in *C++ Should Be C++* (https://wg21.link/P3023R1).

With genericity and simplicity in mind, let us now investigate the proposed `std::maybe_view<T>`. In essence, it "may or may not store a value of type `T` in its storage space". The wording used here is notably *identical* to that of the proposal that introduced optional[1]. This duplication alone is cause for concern due to the complexity of having multiple standardized types representing the same thing.

Indeed, the interfaces of the two types have a substantial amount in common. In the table below, let `V` be some type, `Opt` be either `optional<V>` or `maybe_view<V>`, `o` and `o2` be objects of type `Opt`, and `v` be an object of type `V`:

`std::optional<V>`	**both**	**Proposed** `std::maybe_view<V>`
`Opt(nullopt)`	`Opt()`	
	`Opt(v)`	
	`Opt(in_place, v)`	
`o = nullopt;`	`o = v;`	
`o.emplace(v);`		
`o.reset();`		
`o.swap(o2);`		
	`o == o2` and `o <=> o2`	
	`o == v` and `o <=> v`	
`*o` and `o->m`	`o.transform(f)`	`o.begin()` and `o.end()`
`o.has_value()` and `bool(o)`	`o.and_then(f)`	`o.size()`
`o.value_or(v)`	`o.or_else(f)`	`o.data()`
`std::hash<Opt>{}(o)`		

Nevertheless, Downey argues that the interface differences make this new type worthwhile. Consider each of these differences:

1. **`std::maybe_view` lacks a dereference operator and boolean conversion**. Dereference and boolean conversion are the standard interface to optional-like objects and originate in nullable C pointers. Generic algorithms that today output both a `vector<optional<T>>` and a `vector<shared_ptr<T>>` using optional-like interfaces, will not work with a `vector<maybe_view<T>>`. This interface difference goes against genericity.

2. **`std::maybe_view` contains some, but not all, accessors**. `std::maybe_view` provides the continuation functions `transform`, `and_then`, and `or_else` but not `value_or`. What's the motivation for this omission? A GitHub search[2] results in 111k uses of `value_or`. If we add future accessors to `std::optional`, what will be the basis for deciding whether they should be added to `std::maybe_view`?

3. **`std::maybe_view` 's template parameter supports references**. This is fixing a defect of `std::optional` that is already being fixed with Steve Downey and Peter Sommerlad's `std::optional<T&>` (https://wg21.link/P2988R3) paper.

4. **`std::maybe_view` satisfies the `range` concept**. This interface difference allows `std::maybe_view` to be used in more algorithms than `std::optional`, which is great. However, if this is the only viable interface difference, we should add range support to

`std::optional` to avoid confusing users with two otherwise identical types. This interface difference goes against simplicity.

The standard providing two almost identical types will result in choice overload (https://en.wikipedia.org/wiki/Overchoice), which is mentally draining. Downey suggests using `std::maybe_view` when "the value will have operations applied if present, and ignored otherwise." For a return type, how could one know if this will be the case for a caller? For an argument type, this may be true for one implementation, but what if it changes? Users should not have to spend mental energy considering questions like these.

Survey of existing practice

Since its standardization, `std::optional` 's usage has increased in modern C++ codebases. Iterating over this standard type is a recurring topic as shown in forum discussions such as Iterating over std::optional (https://stackoverflow.com/questions/72812599/iterating-over-stdoptional) and begin and end iterators for std::optional? (https://www.reddit.com/r/cpp/comments/17tyd44/begin_and_end_iterators_for_stdoptional_how_to/).

Some in the C++ community have already experienced this idea. owi_optional (https://github.com/seleznevae/owi_optional) is an open-source implementation. The following code snippet shows an example using syntax similar to our proposal:

```
#include "owi/optional.hpp"

owi::optional<int> empty;
for (int i : empty) {   // iterating over empty optional
    std::cout << i;
}

owi::optional<int> opt{ 123 };
for (int i : opt) { // iterating over non-empty optional
    std::cout << i;
}
```

Several other programming languages with `std::optional` -like types allow direct iteration. Rust's Option (https://doc.rust-lang.org/std/option/#iterating-over-option) type and Scala's Option (https://www.scala-lang.org/api/2.13.5/scala/Option.htm) are notable examples. The following code illustrates a typical Rust usage example:

```
let yep = Some(42);
let nope = None;
// chain() already calls into_iter(), so we don't have to do so
let nums: Vec<i32> = (0..4).chain(yep).chain(4..8).collect();
assert_eq!(nums, [0, 1, 2, 3, 42, 4, 5, 6, 7]);
let nums: Vec<i32> = (0..4).chain(nope).chain(4..8).collect();
assert_eq!(nums, [0, 1, 2, 3, 4, 5, 6, 7]);
```

Proposal

We propose a simple design alternative to P1255R12: make `std::optional` satisfy the `ranges::range` concept where iterating over a `std::optional` object will iterate over its 0 or 1 elements.

To `view` or not to `view`

As with the proposed `std::maybe_view`, `std::optional` should be a `view`. As it only ever has at most one element, it would satisfy the semantic requirements.

The question is *how* to opt `std::optional` into being a `view`. There are two ways to do this:

1. Change `std::optional<T>` to inherit from `std::ranges::view_interface<std::optional<T>>`, or
2. Specialize `std::ranges::enable_view<std::optional<T>>` to `true`.

The problem with inheriting from `ranges::view_interface`, separate from any questions of ABI and the like, are that it would bring in more functions into `std::optional`'s public interface: `empty()`, `data()`, `size()`, `front()`, `back()`, and `operator[]()`. While `empty()` is potentially sensible (if unnecessary given that both `has_value()` and `operator bool` exist), the rest are actively undesirable. Note that all the accessor objects in `std::ranges` will still work (e.g. `std::ranges::size`, `std::ranges::data`, etc.) simply from providing `begin()` and `end()`.

So we propose instead to specialize `ranges::enable_view` (similar to `std::string_view` and `std::span`).

`std::optional<T>` is not a borrowed range, although `std::optional<T&>` (when added) would be.

Choice of type for `iterator`

`optional<T>` is a contiguous, sized range, so the simplest choice of iterator is `T*`. The original revision of this paper simply chose `T*` (and `T const*`) for the iterator types.

However, there are some benefits to having a distinct iterator type: it prevents some misuse and some potential confusion from code of the form:

```
void takes_pointer(T const*);

void f(optional<T> const& o) {
    // what does this mean?
    T const* p = o.begin();

    // do we want this to work?
    takes_pointer(o.begin());

    // or this?
    if (o.begin()) { /* ... */ }
}
```

Users might misinterpret `p` to always point to the storage for the `T` object in the disengaged case and think they might be able to *placement-new* on top of it (even though `begin()` should ideally return `nullptr` for the disengaged case). The call to `takes_pointer` is not really something we intend on supporting. Likewise the conversion to `bool`.

The goal of `begin()` and `end()` is simply to provide the range interface for `optional`, so they should be the *minimal* implementation to provide that functionality without unintentionally introducing other functionality.

As such, a better choice (as discussed in Tokyo) would be to make the `iterator` and `const_iterator` types implementation-defined, where the implementations would hopefully provide some class type to catch many such misuses.

Additional iterator types

Typically, containers provide a large assortment of iterators. In addition to the `iterator` you get from `begin()` / `end()`, there is also a `reverse_iterator` (from `rbegin()` / `rend()`), a `const_iterator` (from `cbegin()` / `cend()`), and a `const_reverse_iterator` (from `crbegin()` / `crend()`). The original revision of this paper provided the full set of iterator algorithms.

However, we think this is unnecessary for the same reason it is unnecessary to provide `data()` and `size()`.

`ranges::cbegin()` and `ranges::cend()` will already work and correctly provide `const_iterator`, so there is no benefit to providing it directly. Generic code should already be using those function objects rather than attempting to call member `cbegin()` and `cend()` directly.

Similarly, `ranges::rbegin()` and `ranges::rend()` will already work and correctly provide a `reverse_iterator`. Indeed, for all containers in the standard library, member `rbegin` and `rend` already do the same thing that `ranges::rbegin` and `ranges::rend` would do anyway.

Interestingly, in this particular case, `optional` could do better. Since our size is at most `1`, reversing an `optional` is a no-op: `rbegin()` could simply be implemented to return `begin()` as an optimization. However, given that our `iterator` is contiguous, the added overhead of `std::reverse_iterator` will get optimized out anyway.

Given that neither `cbegin` nor `rbegin` add value, `crbegin` certainly doesn't either.

We therefore propose to provide the minimal interface to opt into ranges: simply `begin()` and `end()`. All the functionality is there for the user if they want to use the `ranges::` objects.

Wording

[optional.syn] **(https://eel.is/c++draft/optional.syn)**

Add the specialization of `ranges::enable_view` and `format_kind` to `[optional.syn]` right after the `class optional` declaration:

```
namespace std {
  // [optional.optional], class template optional
  template<class T>
    class optional;                                      // partially freestanding

+ template<class T>
+   constexpr bool ranges::enable_view<optional<T>> = true;
+ template<class T>
+   constexpr auto format_kind<optional<T>> = range_format::disabled;

  template<class T>
    concept is-derived-from-optional = requires(const T& t) {  // exposition only
        []<class U>(const optional<U>&){ }(t);
    };
  // ...
}
```

[optional.optional.general] (https://eel.is/c++draft/optional.optional.general)

Add:

- the iterator types right after definition of `value_type`
- the range accessors to `[optional.optional.general]` right before the `observers`

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type             = T;
+   using iterator               = implementation-defined; // see [optional.iterator
+   using const_iterator         = implementation-defined; // see [optional.iterator

    // ...

    // [optional.swap], swap
    constexpr void swap(optional&) noexcept(see below);


+   // [optional.iterators], iterator support
+   constexpr iterator begin() noexcept;
+   constexpr const_iterator begin() const noexcept;
+   constexpr iterator end() noexcept;
+   constexpr const_iterator end() const noexcept;

    // [optional.observe], observers
    constexpr const T* operator->() const noexcept;
    // ...
  };
}
```

[optional.iterators]

Add a new clause `[optional.iterators]` between [optional.swap] (https://eel.is/c++draft/optional.swap) and [optional.observe] (https://eel.is/c++draft/optional.observe):

Iterator support [optional.iterators]

```
using iterator       = implementation-defined;
using const_iterator = implementation-defined;
```

These types model contiguous_iterator ([iterator.concept.contiguous]), meet the Cpp17RandomAccessIterator requirements ([random.access.iterators]), and meet the requirements for constexpr iterators ([iterator.requirements.general]), with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator` and `const T&` for `const_iterator`.

All requirements on container iterators ([container.reqmts]) apply to `optional::iterator` and `optional::const_iterator` as well.

Any operation that initializes or destroys the contained value of an optional object invalidates all iterators into that object.

```
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
```

Returns: If `has_value()` is true, an iterator referring to the contained value. Otherwise, a past-the-end iterator value.

```
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
```

Returns: `begin() + has_value()`.

[version.syn] **(https://eel.is/c++draft/version.syn)**

Add the following macro definition to `[version.syn]`, header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
    #define __cpp_lib_optional 202110L // also in <optional>
+   #define __cpp_lib_optional_range_support 20XXXXL // freestanding, also in <optio
    #define __cpp_lib_out_ptr 202106L // also in <memory>
```

LEWG feedback

P3168R0

P3168R0 was reviewed during the LEWG sessions at Tokyo 2024. It was decided to proceed with our proposed solution, albeit with a few changes.

Tokyo 2024 P3168R0 related polls:

1. We want to make optional satisfy the range concept.

SF	F	N	A	SA
11	8	2	3	0

- Attendance: `20 + 9`
- # of Authors: `4`
- Authors' position: `3 x SF, WF`
  Outcome: `Consensus in favour`

2. > Modify P3168R0 (Give std::optional Range Support) by opting out of formatter range support for optional and changing the return type of iterator functions from T* to iterator and family, and then send the revised paper to LWG for C++26 classified as B2, to be confirmed with a Library Evolution electronic poll.

SF	F	N	A	SA
11	3	1	0	2

- Attendance: `19 + 5`
- # of Authors: `1`
- Authors' position: `SF`
  Outcome: `Consensus in favour`

P3168R1

We applied the LEWG feedback from Tokyo into P3168R1. This revision was sent for *2024-04 Library Evolution Polls (P3213R0)* (https://wg21.link/P3213R0). The results published in *2024-04 Library Evolution Poll Outcomes (P3214R0)* (https://wg21.link/P3214R0) revealed an expected consensus into forwarding this proposal to LWG.

> Send "[P3168R1] Give std::optional range support" to Library Working Group for C++26.

SF	F	N	A	SA
10	3	1	2	3

Implementation experience

Initial experiments

To evaluate the effectiveness of the proposed `std::optional` modifications, we conducted extensive testing across two multi-million line internal Adobe repositories. This rigorous testing process revealed no build or test suite regressions, indicating a smooth integration of the modifications with existing codebases.

All future experiments and developments related to these modifications will be conducted within the *Beman Project* (https://www.bemanproject.org/), ensuring a centralized approach for continuous integration and iterative improvements.

Beman.Optional26

P3168R1 is fully implemented within the *Beman Project* (https://www.bemanproject.org/). The implementation, named Beman.Optional26 (https://github.com/beman-project/Optional26), aims to provide a comprehensive and standalone library that adheres to the proposal's specifications. This ongoing work demonstrates a commitment to ensuring the robustness and applicability of the `std::optional` enhancements in real-world scenarios.

Dependencies

In the implementation of iterators for `Beman.Optional26`, we utilized the boost/stl_interfaces (https://github.com/boostorg/stl_interfaces) library, which implements Zach Laine's proposal for P2727R4: std::iterator_interface (https://wg21.link/P2727R4). This choice was made to adhere to modern standards and ensure compatibility with proposed C++ features.

Currently, `Beman.Optional26` includes a subset of headers copied locally. However, for long-term sustainability, future iterations may consider integrating `stl_interfaces` as a direct dependency or implementing additional C++26-related proposals within the Beman library ecosystem. This approach aims to foster code reuse, maintain compliance with evolving standards, and facilitate broader community adoption.

std::optional extensions

Additionally, the `Beman.Optional26` library aims to support and integrate other upcoming C++26 features related to `std::optional`. For instance, it already includes implementation for P2988R5: std::optional<T&> (https://wg21.link/P2988R5), authored by Steve Downey and Peter Sommerlad. These efforts are geared towards expanding functionality and aligning with the latest developments in the C++ standard.

Compiler support

We have tested and successfully compiled Beman.Optional26 (https://github.com/beman-project/Optional26) with C++20, C++23 and C++26 on the latest versions of GCC (13, 14, trunk) and Clang (17, 18, 19, trunk), running on Ubuntu 24.04 (x86_64 and arm64) or Compiler Explorer (https://godbolt.org/). This ensures compatibility and performance across a range of modern development environments.

Future integrations will test compatibility with the latest versions of MSVC and other major compilers, ensuring broad support across different platforms and toolchains.

Integration with Compiler Explorer

We have made an `alpha version` of the `Beman.Optional26` library available on Compiler Explorer (https://godbolt.org/), allowing developers to test and experiment with the new features directly in an interactive environment. This provides an accessible way for the community to engage with the implementation and offer feedback for further improvements.

To use the `Beman.Optional26` library inside Compiler Explorer, you need to create a default C++ entry, select `Current libraries`, search for `Beman.Optional26`, select the version (e.g., `trunk`), close the window, and try to compile.

Examples

All examples related to P3168 can be found in examples/README.md (https://github.com/beman-project/Optional26/tree/main/examples/README.md) or in tests (optional_range_support.t.cpp (https://github.com/beman-project/Optional26/tree/main/src/Beman/Optional26/tests/optional_range_support.t.cpp)).

Example: *std::optional* vs *beman::optional26::optional*

The following code snippet demonstrates a basic usage of Beman's optional compared with std::optional (https://godbolt.org/z/ds5MvfGe6):

```
#include <optional>
#include <Beman/Optional26/optional.hpp>

// empty optional example
std::optional<int> std_empty_opt;
beman::optional26::optional<int> beman_empty_opt;
assert(!std_empty_opt && !beman_empty_opt);

// optional with value example
std::optional<int> std_opt = 26;
beman::optional26::optional<int> beman_opt = 26;
assert(std_opt && beman_opt);
assert(*std_opt == *std_opt);
```

Example: range loop

This example demonstrates a basic range loop over Beman's optional (https://godbolt.org/z/f8dWaxsGo):

```
#include <Beman/Optional26/optional.hpp>

// iterate over empty optional example
beman::optional26::optional<int> empty_opt{};
for ([[maybe_unused]] const auto& i : empty_opt) {
    // Should not see this in console.
    std::cout << "\"for each loop\" over C++26 optional: empty_opt\n";
}

// iterate over optional with value example
beman::optional26::optional<int> opt{26};
for (const auto& i : opt) {
    // Should see this in console.
    std::cout << "\"for each loop\" over C++26 optional: opt = " << i << "\n";
}
```

Example: Pythagorean triples

The `Beman.Optional26` library enables the support for complex examples, such as generating an infinite sequence of Pythagorean triples (https://godbolt.org/z/fGr8jYM6P).

Appendix: Comparison P1255R12 vs P3168R0

This section compares examples from P1255R12 to equivalents using our own proposal.

Optional access

P1255R12	This Proposal

`for (auto&& opt : views::nullable(get_optional())) {`
`  // a few dozen lines...`
`  use(opt); // opt is safe`
`}`
	`for (auto&& opt : get_optional()) {`
	`  // a few dozen lines...`
	`  use(opt); // opt is safe`
	`}`

Optional assignment

P1255R12	This Proposal

`auto opt_int = get_optional();`
`for (auto&& i : views::nullable(std::ref(opt_int))) {`
`  // a few dozen lines...`
`  i = 123;`
`}`
	`auto opt_int = get_optional();`
	`for (auto&& i : opt_int) {`
	`  // a few dozen lines...`
	`  i = 123;`
	`}`

Range chain example (1)

```
vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1};
auto test = [](int i) -> optional<int> {
    switch(i) {
        case 1:
        case 3:
        case 7:
        case 9:
            return i;
        default:
            return {};
    }
};
```

P1255R12	This Proposal

`auto&& r = v`
`  | views::transform(test)`
`  | views::transform(views::nullable)`
`  | views::join |`
`  | views::transform(`
`  [](int i) { cout << i; return i; }`
`  );`
	`auto&& r = v`
	`  | views::transform(test)`
	`  | views::filter([](auto x) { return bool(x); })`
	`  | views::transform([](auto x){ return *x; })`
	`  | views::transform(`
	`  [](int i) { cout << i; return i; }`
	`  );`

```
for (auto&& i : r) {
    std::cout << i << "\n";
}
```

Range chain example (2)

```
unordered_set<int> set{1, 3, 7, 9};
auto flt = [=](int i) -> optional<int> {
    if (set.contains(i))
        return i;
    else
        return {};
};
```

P1255R12	This Proposal

`for(auto i : views::iota{1,10} | views::transform(flt)) {`
`  for (auto j : views::nullable(i)) {`
`  for (auto k : views::iota(0, j))`
`  cout <<'\a';`
`  cout << '\n';`
`  }`
`}`
	`for(auto i : views::iota{1,10} | views::transform(flt`
	`  for (auto j : i) { // no need to transform`
	`  for (auto k: views::iota(0, j))`
	`  cout <<'\a';`
	`  cout << '\n';`
	`  }`
	`}`

yield_if

Eric Niebler's Pythagorian Triples (https://ericniebler.com/2018/12/05/standard-ranges/) triple example:

P1255R12	This Proposal

`auto yield_if = []<class T>(bool b, T x) {`
`  return b ?`
`  maybe_view<T>{move(x)} :`
`  maybe_view<T>{};`
`};`
	`auto yield_if = []<class T>(bool b, T x) {`
	`  return b ?`
	`  optional<T>{move(x)} :`
	`  nullopt;`
	`};`

References

- Fernando Cacciola and Andrzej Krzemieński. N3793: A proposal to add a utility class to represent optional objects (Revision 5). https://wg21.link/N3793 (https://wg21.link/N3793), 10/2013.
- James C. Dehnert and Alexander Stepanov. Fundamentals of Generic Programming. Lecture Notes in Computer Science volume 1766. http://stepanovpapers.com/DeSt98.pdf (http://stepanovpapers.com/DeSt98.pdf), 1998.
- Steve Downey. P1255R12: A view of 0 or 1 elements: views::maybe. https://wg21.link/p1255r12 (https://wg21.link/p1255r12), 01/2024.
- Steve Downey and Peter Sommerlad. P2988R3: std::optional<T&>. htts://wg21.link/P2988R3, 02/2024.
- Hinnant, Howard et al. "Direction for ISO C++." 15 October 2022, https://wg21.link/P2000R4 (https://wg21.link/P2000R4).
- Eric Niebler et al. P0896R4: The One Ranges Proposal. https://wg21.link/P0896R4 (https://wg21.link/P0896R4), 11/2018.
- David Sankel. P3023R1: C++ Should Be C++. https://wg21.link/P3023R1 (https://wg21.link/P3023R1), 10/2023.

- Wikipedia Foundation. Overchoice. https://en.wikipedia.org/wiki/Overchoice (https://en.wikipedia.org/wiki/Overchoice), 27 February 2024.
- Beman Project. https://www.bemanproject.org/ (https://www.bemanproject.org/)
- Beman.Optional26. https://github.com/beman-project/Optional26 (https://github.com/beman-project/Optional26)
- Boost/stl_interfaces. https://github.com/boostorg/stl_interfaces (https://github.com/boostorg/stl_interfaces)
- Zach Laine. P2727R4: std::iterator_interface. https://wg21.link/P2727R4 (https://wg21.link/P2727R4), 18 June 2022.

1. See the introduction of *A proposal to add a utility class to represent optional objects* (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html) ↩️

2. See GitHub search of '".value_or" path:*.cpp' (https://github.com/search?q=%22.value_or%22+path%3A*.cpp&type=code) ↩️